
Final Report – Applied Project 1: Deep RL

Joelle Alachkar
Student ID: 391740
EPFL

Omar Shibli
Student ID: 408339
EPFL

Andrea Tarabay
Student ID: 391738
EPFL

Abstract

This project implements and evaluates four deep reinforcement learning algorithms: DQN, PPO, SAC, and TD3. The methods are tested on Gym environments with both discrete and continuous action spaces, including CartPole, Acrobot, Pendulum, and Mountain Car. We compare their learning performance, stability, sample efficiency, computational cost, and suitability for different types of control tasks.

1 Introduction

Deep reinforcement learning (DRL) combines reinforcement learning with neural-network function approximation, allowing reinforcement-learning methods to scale to problems with large and complex state spaces. This project studies four common DRL algorithms: DQN, PPO, SAC, and TD3, which cover three major algorithm families: value-based methods, policy-gradient methods, and actor-critic methods. The goal is to implement these algorithms from scratch and test them on standard Gym control tasks. CartPole and Acrobot are used for discrete control, while Pendulum and Mountain Car are used for continuous control. The algorithms are then compared based on convergence, stability, sample efficiency, computational cost, and sensitivity to implementation choices.

2 Background and Related Work

2.1 Deep Q-Networks (DQN) (1)

DQN extends Q-learning by using a neural network to approximate the action-value function $Q(s, a)$. Given a state, the network outputs one Q-value for each discrete action. The policy is therefore implicit: the agent selects the action with the highest predicted Q-value.

The main challenge in DQN is training stability. To reduce correlation between consecutive samples, DQN uses experience replay, where past transitions are stored and sampled randomly during training. It also uses fixed Bellman targets during each gradient update, preventing gradients from being backpropagated through the target values.

2.2 Proximal Policy Optimization (PPO) (2)

PPO is an on-policy actor-critic algorithm. It uses two networks: an actor, which represents the policy $\pi_\theta(a | s)$, and a critic, which estimates the value function $V(s)$. The critic is used to compute advantage estimates, which indicate whether an action was better or worse than expected.

The main goal of PPO is to improve the policy without changing it too much after one batch of collected data. To do this, PPO compares the new policy to the old policy using the probability ratio $r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$. This ratio is then clipped, so that policy updates remain small and stable. The resulting clipped loss increases the probability of actions with positive advantage and decreases the probability of actions with negative advantage, while avoiding overly large updates. The final PPO objective combines three terms: the clipped policy loss for updating the actor, a value-function loss for training the critic, and an entropy bonus to encourage exploration.

2.3 Off-Policy Actor-Critic Methods: SAC and TD3

SAC (3) and TD3 (4) are both off-policy actor-critic algorithms for continuous control. Both use an actor to produce actions, two critics to estimate $Q(s, a)$, a replay buffer storing $(s_t, a_t, r_t, s_{t+1}, d_t)$, and slowly updated target networks using $\theta' \leftarrow \tau\theta + (1 - \tau)\theta'$. Both also reduce overestimation bias by using the minimum of two critic estimates. The purpose of using two critics is to make the value target more conservative, since one critic may overestimate the value of an action due to approximation errors. By taking the smaller estimate, the algorithm reduces the chance that the actor is trained toward actions that only appear good because of critic error.

The main difference is in the policy and exploration strategy. SAC uses a stochastic policy and adds an entropy term to the objective, written as $\sum_t \mathbb{E} [r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t))]$, where α controls the trade-off between reward maximization and exploration. Thus, SAC encourages exploration through entropy regularization.

TD3 instead uses a deterministic policy $a = \pi_\phi(s)$, and improves stability using delayed actor updates and target policy smoothing. In target policy smoothing, the target action is computed as $\tilde{a} = \pi_{\phi'}(s') + \epsilon$, where $\epsilon \sim \text{clip}(\mathcal{N}(0, \sigma), -c, c)$. Its Bellman target is $y = r + \gamma \min_{i=1,2} Q_{\theta'_i}(s', \tilde{a})$. Therefore, SAC relies on stochastic entropy-based exploration, while TD3 relies on deterministic control with smoothed targets and delayed policy updates.

3 Approach

For DQN, we adapt the original value-based algorithm to low-dimensional Gym environments with discrete actions. Since the observations are compact state vectors, no image preprocessing or convolutional network is needed. The Q-function is represented by a fully connected neural network that outputs one Q-value per action. Actions are selected with an ϵ -greedy policy, transitions are stored in a replay buffer, and the network is trained from random minibatches using the Bellman target. To improve stability, training starts only after an initial set of transitions has been collected, and the Bellman target is treated as fixed during each update.

For PPO, we use the clipped actor-critic formulation. The actor represents the policy, while the critic estimates the value function used to compute advantages. For discrete environments, the actor outputs a categorical distribution. For continuous control, it outputs the mean of a Gaussian policy. PPO collects rollout batches using the current policy, computes advantages with Generalized Advantage Estimation, and then performs several epochs of minibatch optimization. The objective combines a clipped policy loss, a value-function loss, and an entropy term. The clipping mechanism limits how much the policy can change after each rollout, which improves training stability.

For the continuous control environments, we implemented two similar off-policy actor-critic methods: SAC and TD3. Both algorithms store transitions in a replay buffer which allows previously collected data to be reused for multiple gradient updates.

The **SAC** implementation follows the original actor-critic formulation with maximum-entropy. The network outputs the mean and log standard deviation of a Gaussian distribution and applies tanh transformation to keep the action within environment bounds. The log probability also accounts for tanh correction in order to get correct entropy regularization. SAC also uses two Q-networks in order to reduce overestimation bias by taking the minimum of the two. A value network and target value network are used to compute stable soft Bellman targets, with the target network updated by Polyak averaging.

TD3 differs by using a deterministic off-policy actor-critic method rather than Gaussian. The actor maps each state directly to one continuous action, and also uses two Q-networks for the same reason. TD3 uses clipped double Q-learning, delayed actor updates, target policy smoothing, and the actor is updated less frequently than the critics, and clipped Gaussian noise is added to target actions to avoid overfitting the critic to sharp value peaks.

4 Experimental Setup

The algorithms are evaluated on four OpenAI Gym environments:

- **CartPole-v1**: discrete-action balancing task.
- **Acrobot-v1**: discrete-action swing-up task with sparse negative rewards.
- **Pendulum-v1**: continuous-control torque optimization task.
- **MountainCarContinuous-v0**: continuous-control task where a car must build momentum to reach the top of a hill.

Each experiment is trained for 100,000 environment steps and run with three random seeds. Performance is reported using averaged episode returns across seeds. DQN is evaluated on the discrete environments, PPO is evaluated on both discrete and continuous environments, and SAC and TD3 are used for continuous-control experiments. For each algorithm, we first perform a grid search to select the final hyperparameter configuration, and then study sensitivity by varying key hyperparameters one at a time.

4.1 DQN

For DQN, the Q-network is an MLP with hidden dimensions (128, 128, 256) and ReLU activations. Training uses RMSProp, minibatches of size 32, a replay buffer, and an ϵ -greedy exploration schedule annealed from 1.0 to 0.1. Evaluation is performed every 5,000 steps over 5 episodes using a small fixed exploration rate $\epsilon = 0.05$.

4.2 PPO

For PPO, the actor and critic are separate MLPs with two hidden layers of 64 units and $\tanh$ activations. Each rollout contains 2,048 environment steps collected with 4 parallel actors. Advantages are computed with $\gamma = 0.99$ and $\lambda = 0.95$. PPO uses clipping parameter $\epsilon = 0.2$, Adam with learning rate 2.5×10^{-4} , minibatches of size 256, 10 update epochs, gradient norm clipping at 0.5, and linear learning-rate annealing. Evaluation is performed every 5,000 steps over 10 deterministic episodes.

4.3 SAC

SAC was evaluated on the continuous environments Pendulum-v1 and MountainCarContinuous-v0. For pendulum, the configuration used 3 seeds, a replay buffer of size 200,000, minibatch size of 256, a discount factor $\gamma = 0.99$, learning rate 3×10^{-4} , and an entropy term of 0.2. The agent was trained for 30,000 environment steps, which is less than our other algorithms since it converged faster, and it was evaluated every 1,000 steps.

For MountainCarContinuous-v0, the final SAC experiment was modified since vanilla SAC converged to an inaction policy. The final configuration uses similar parameters but the change was related to adding a momentum heuristic to collect useful trajectories. Reward shaping used position progress and velocity terms during training, while the evaluation return and success rate were computed using the original MountainCarContinuous environment. This distinction is important because average return alone can be misleading in this environment: a near-zero return may correspond to doing nothing rather than solving the task.

4.4 TD3

For TD3, the actor is an MLP with hidden layers of size 400 and 300, ReLU activations, and a final $\tanh$ output scaled to the environment action bounds. The two critics have the same MLP structure and estimate scalar state-action values $Q_{\theta_1}(s, a)$ and $Q_{\theta_2}(s, a)$. We used Adam for both actor and critic optimization, with learning rates of 10^{-3} , batch size 100, discount factor $\gamma = 0.99$, replay buffer size 10^6 , and target update rate $\tau = 0.005$. Exploration noise was sampled from $\mathcal{N}(0, 0.1)$, while target policy smoothing used noise $\mathcal{N}(0, 0.2)$ clipped to $[-0.5, 0.5]$. The actor was updated every $d = 2$ critic updates. Since TD3 is designed for continuous actions, it was evaluated on Pendulum-v1 and MountainCarContinuous-v0.

5 Results

5.1 Discrete Control

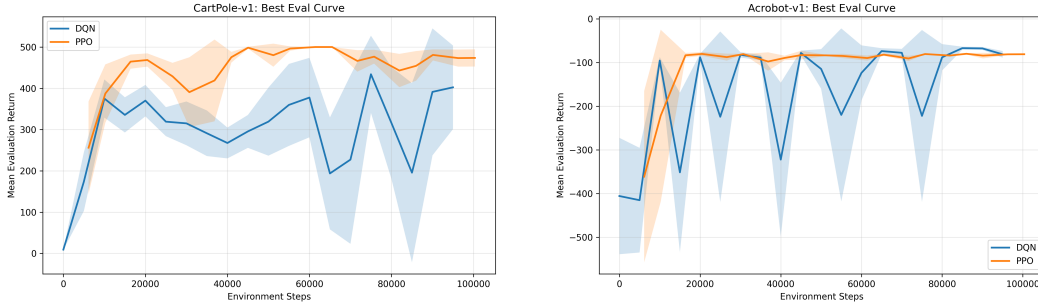
5.1.1 CartPole-v1

CartPole-v1 is a dense-reward balancing task where the maximum episode return is 500. Figure 1 shows that both DQN and PPO improve during training, but PPO performs more reliably. PPO quickly reaches returns close to the maximum score and remains stable for most of training. DQN also improves early, but its evaluation return fluctuates much more and stays below PPO by the end of training. This instability is likely caused by errors in the learned Q-values: since DQN chooses the action with the highest predicted Q-value, small value-estimation errors can lead to different greedy actions and unstable returns. Overall, PPO achieves both higher final performance and smoother convergence, suggesting that its clipped policy updates are particularly effective on this dense-reward control task.

5.1.2 Acrobot-v1

Acrobot-v1 gives a reward of -1 at each time step until the goal is reached, so less negative returns mean that the agent solves the task in fewer steps. Figure 1 shows that both algorithms eventually reach returns around -80 , indicating that they learn successful swing-up policies.

The main difference is stability. After reaching good performance, PPO remains in a narrow range around -80 throughout training. In contrast, DQN exhibits several large drops in performance and substantially higher variance across seeds. This is consistent with the instability of DQN discussed in Section 5.1.1. Overall, although both algorithms eventually reach similar returns, PPO achieves considerably more stable learning on this task.



(a) CartPole-v1 evaluation performance averaged over three seeds.

(b) Acrobot-v1 evaluation performance averaged over three seeds.

Figure 1: Discrete-control evaluation performance for DQN and PPO.

5.2 Continuous

5.2.1 Pendulum

Pendulum-v1 was used to compare the three continuous-control algorithms: PPO, SAC, and TD3. Since the environment gives negative rewards, higher returns, meaning less negative values, indicate better performance.

PPO showed clear improvement on Pendulum-v1, but its learning was more gradual than SAC and TD3. Its average evaluation return started around -240 and initially remained low because PPO first collects rollout data before performing policy updates. Since PPO is on-policy, it cannot update continuously from a replay buffer like SAC or TD3. After several rollout and update phases, PPO improved to approximately the range of -190 to -170 , showing that it learned a reasonable continuous-control policy, although its final performance remained below SAC.

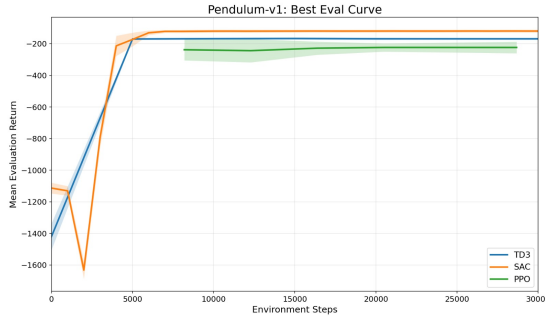
TD3 also performed well on Pendulum-v1. Starting from around -1421 , it improved rapidly and reached approximately -170 by about 15,000 steps, after which it remained stable. This shows that

TD3 is effective for dense-reward continuous-control tasks, although it converged slightly slower and to a lower final return than SAC.

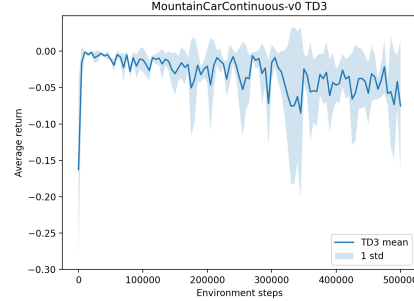
SAC achieved the fastest convergence and the best final return on Pendulum-v1. Although it started with a low return of around -1113 , it improved sharply within the first few thousand steps, reaching about -213 by 4,000 steps and approximately -131 by 6,000 steps. It then stabilized close to -120 , showing that SAC learned an effective policy very early in training.

Overall, SAC outperformed both PPO and TD3 on Pendulum-v1. Compared with PPO, SAC benefited from being an off-policy method. Instead of collecting a rollout buffer and then discarding old data after the update phase, SAC stores transitions in a replay buffer and reuses them for many gradient updates. This makes SAC more sample-efficient, especially in continuous-control tasks where the agent must learn both the direction and magnitude of the torque.

SAC also performed better than TD3 because of its stochastic maximum-entropy objective. TD3 uses a deterministic actor and relies on added exploration noise, while SAC maintains a stochastic policy and explicitly encourages exploration through the entropy term. This helps SAC explore useful torque actions early while still converging toward high-return behavior. As a result, SAC reached strong performance faster and achieved the best final return. Therefore, the Pendulum-v1 results show that replay-buffer reuse and entropy-based exploration made SAC the most effective algorithm for this task.



(a) Pendulum-v1 evaluation performance.



(b) MountainCarContinuous-v0 evaluation performance.

Figure 2: Continuous-control evaluation performance for TD3, SAC, and PPO.

5.2.2 Mountain Car

MountainCarContinuous-v0 was the most challenging continuous-control environment in our experiments. Unlike Pendulum-v1, where the reward gives dense feedback at each step, MountainCarContinuous has a sparse success signal in which the agent receives a large reward only after reaching the goal. If the agent does not reach this goal, the main signal is the action penalty. This creates a local optimum where the agent learns to apply no force in both TD3 and SAC. Figure 2b shows the results of the failed run on TD3.

We noticed this issue when the learning curve improved quickly from slightly negative to values near zero, which initially seems like progress, but after looking at the task definition and success rate, we found that the policy was not reaching the goal.

To solve this, we introduced a momentum-based SAC setup. The idea was to help SAC discover successful trajectories early enough for the replay buffer to contain the goal reward. During the first 10,000 environment steps, the agent pushed in the direction of the car’s current velocity so that the car could build enough momentum to climb the hill. SAC updates were delayed until after this warm-start phase, preventing the critic from being trained only on unsuccessful inaction trajectories.

With this setup, SAC solved MountainCarContinuous over three seeds with a final mean return of 92.63 with a standard deviation of 0.2 and a success rate of 1.0. This confirms that the agent was no longer exploiting the near-zero inaction policy. Instead, it learned a policy that consistently reached the goal under the original environment evaluation. The results can be seen in Figure 3a in which SAC reaches the goal.

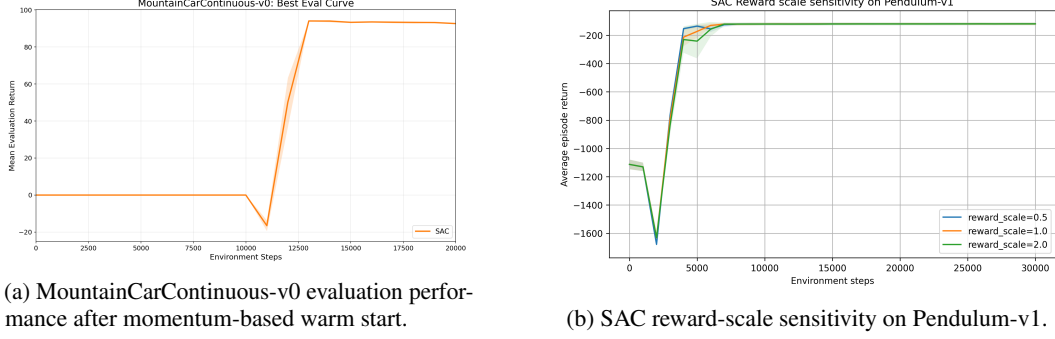


Figure 3: Continuous-control results for MountainCarContinuous-v0 and SAC reward-scale sensitivity.

5.3 Hyperparameter Sensitivity

DQN The strongest sensitivities were the learning rate and discount factor, as shown in Figure 4.

The learning rate mainly controls stability: values that are too large cause sharp performance drops, because aggressive Q-network updates can amplify inaccuracies in the bootstrapped targets. The discount factor controls how much future rewards matter. If γ is too low, DQN mostly values immediate rewards and ignores rewards that come later. This hurts Acrobot, where early swing actions have similar immediate rewards but different long-term effects, and CartPole, where actions must be evaluated by how long they keep the pole balanced in the future.

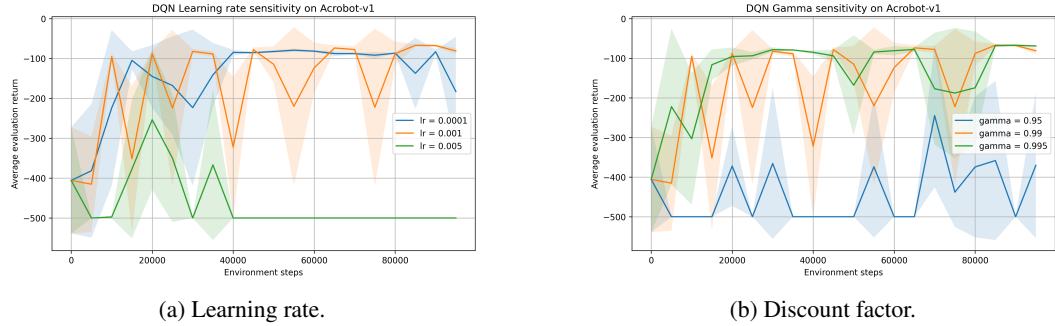


Figure 4: Main DQN hyperparameter sensitivities on Acrobot-v1.

PPO It was most sensitive to the number of update epochs and the learning rate. A smaller learning rate learned more slowly but produced smoother final performance, while larger learning rates improved faster but showed higher variance. The number of update epochs had the clearest effect since when using 10 epochs the learning speed and sample efficiency were improved compared with fewer epochs, while the clipped objective prevented harmful over-updating.

SAC It was our most robust algorithm with respect to hyperparameters. We tested the learning rate, entropy coefficient α , reward scale, and target update coefficient τ . Most configurations resulted in similar final returns once the early learning phase was complete, with a reward scale of 2.0 giving the best final mean return. In Figure 3b we can see the similar results with different reward scaling. This suggests that for dense, low-dimensional continuous-control tasks such as Pendulum-v1, SAC is fairly stable as long as the main implementation details are correct.

TD3 We studied three TD3-specific hyperparameters on Pendulum-v1: policy update frequency, target policy noise, and exploration noise. Figure 6 shows the two most informative experiments: policy update frequency and exploration noise.

Policy update frequency mainly affected the learning speed, where updating the actor every step converged fastest ($d = 1$), and it got slower as we increased d . Exploration noise mainly affected stability with smaller values producing smoother curves and larger noise causing more fluctuations. Target policy noise had a weaker effect and is shown in the appendix. Overall, TD3 was fairly robust

on Pendulum-v1, with hyperparameters mostly affecting learning speed and variance rather than final performance.

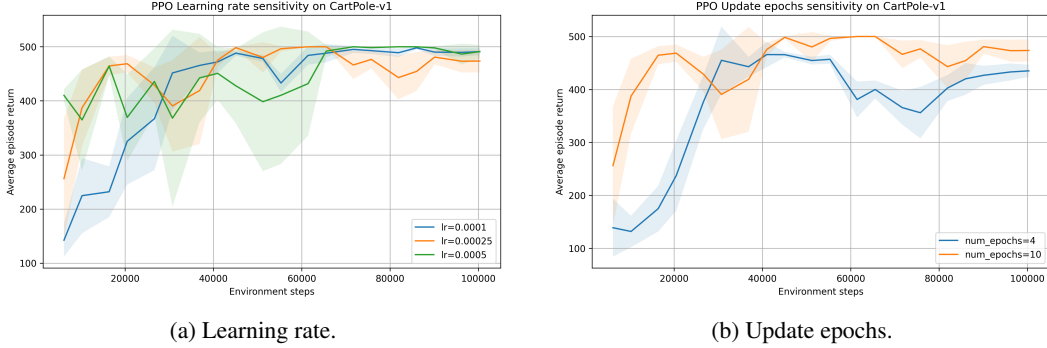


Figure 5: PPO hyperparameter sensitivity experiments on CartPole.

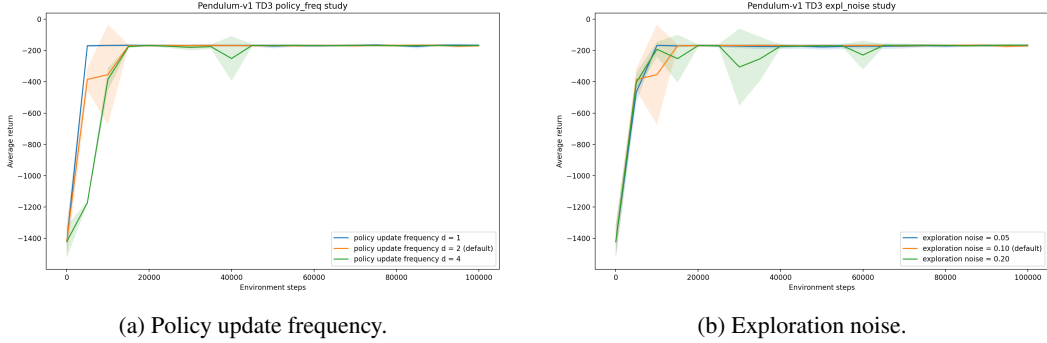


Figure 6: TD3 hyperparameter sensitivity results on Pendulum-v1.

6 Qualitative Discussion

Beyond final evaluation returns, the algorithms differ in their computational cost, policy representation, ability to handle continuous actions, and use of off-policy data. These differences help explain why each algorithm performed better in some environments than others.

Criterion	Best / strongest	Weakest	Main reason
Computational cost	DQN cheapest	SAC most expensive	SAC uses a stochastic actor, two critics, value/target networks, entropy terms, and log-probability corrections.
Policy compactness	TD3 for continuous control; DQN for discrete control	SAC slightly less compact	TD3 uses a deterministic actor, while SAC outputs Gaussian distribution parameters.
Continuous actions	SAC and TD3	DQN	DQN requires discrete actions, while SAC and TD3 directly learn continuous-action policies.
Off-policy data use	SAC	PPO	SAC reuses replay-buffer data while maintaining stochastic entropy-based exploration; PPO is on-policy.

Table 1: Qualitative comparison of the implemented RL algorithms.

Computational cost per iteration. DQN is the cheapest algorithm per update because it only trains one Q-network using minibatches from a replay buffer. PPO is more expensive at the iteration level because it first collects a rollout batch and then performs several epochs of actor-critic optimization on

that batch. TD3 is also more expensive than DQN because it maintains an actor, two critics, and target networks, but its delayed policy updates reduce the cost of updating the actor at every step. SAC was the most computationally expensive in our implementation because it uses a stochastic actor, two Q-networks, a value network, a target value network, entropy regularization, and the tanh log-probability correction. Overall, the approximate computational cost ranking is $DQN < TD3 < PPO \approx SAC$. However, this higher cost is not wasted: in SAC, the extra computation supports entropy-based exploration and more sample-efficient continuous-control learning which helped it achieve the best continuous-control results.

Policy compactness. DQN stores the policy compactly for discrete actions because the policy is simply obtained by selecting the action with the highest Q-value. However, this representation does not naturally extend to continuous actions, since the network would need to compare infinitely many possible actions. For continuous control, TD3 has the most compact deployed policy because its deterministic actor directly maps a state to one action. PPO and SAC also only need the actor during evaluation, but their continuous policies are usually stochastic. SAC is slightly less compact than TD3 because it outputs both the mean and log standard deviation of a Gaussian distribution. Thus, DQN is compact for discrete control, while TD3 is the most compact option for continuous-control deployment.

Scaling to continuous actions. DQN does not scale naturally to continuous actions because it outputs one Q-value per discrete action. PPO can handle continuous control with a Gaussian policy, but its on-policy updates make learning more gradual because fresh rollout data is needed before each update. TD3 scales better by directly learning a deterministic continuous actor, which worked well on dense-reward tasks such as Pendulum-v1. SAC performed best because it combines continuous actor-critic learning with stochastic maximum-entropy exploration, allowing it to explore useful torque values while still converging to a strong policy.

Use of off-policy data. DQN, TD3, and SAC all use replay buffers, so they can reuse past transitions for several updates. This improves sample efficiency compared with PPO, which discards old rollout data after the update phase. DQN uses replay effectively in discrete environments, but it remains sensitive to bootstrapping errors because the policy depends directly on learned Q-values. TD3 also benefits from replay-buffer reuse, but its deterministic actor makes it more dependent on the quality of exploration noise. If the replay buffer does not contain useful successful trajectories, as observed in MountainCarContinuous-v0, TD3 can learn a conservative policy instead of solving the task. SAC made the most effective use of off-policy data because it combines replay-buffer reuse with stochastic entropy-based exploration. This means it not only reuses past data efficiently, but also tends to collect more diverse transitions during early training.

7 Conclusion

This project compared DQN, PPO, SAC, and TD3 across discrete and continuous-control environments. The results show that no algorithm is universally best; performance depends on the action space, reward structure, and exploration difficulty.

For discrete-control tasks, PPO was the most stable overall. DQN learned good policies, but it was more sensitive to learning rate, discount factor, and bootstrapped Q-value errors. PPO reduced these instabilities through its clipped policy update, which limits overly large policy changes.

For continuous-control tasks, SAC and TD3 were more suitable than DQN and generally more sample-efficient than PPO. TD3 worked well on dense continuous-control tasks such as Pendulum-v1, but depended more strongly on exploration quality. SAC achieved the strongest continuous-control performance because it combined replay-buffer reuse with entropy-based stochastic exploration, leading to faster learning and better final returns.

Overall, DQN is suitable for simple discrete tasks, PPO is strong for stable policy optimization, TD3 is effective for compact deterministic continuous-control policies, and SAC is the best choice when sample efficiency and exploration in continuous action spaces are most important.

References

- [1] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller. *Playing Atari with Deep Reinforcement Learning*. arXiv:1312.5602, 2013.
- [2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. *Proximal Policy Optimization Algorithms*. arXiv:1707.06347, 2017.
- [3] Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *Proceedings of the 35th International Conference on Machine Learning*, PMLR 80, 2018.
- [4] Fujimoto, S., van Hoof, H., and Meger, D. Addressing function approximation error in actor-critic methods. *Proceedings of the 35th International Conference on Machine Learning*, PMLR 80, 2018.

A Additional Hyperparameter Sensitivity Results

This appendix presents the remaining hyperparameter sensitivity experiments that were omitted from the main text due to space constraints. Each experiment varies a single hyperparameter while keeping the remaining configuration fixed.

A.1 DQN on Acrobot-v1

Figure 7 shows that DQN is sensitive to several hyperparameters, although the effect is weaker than for the learning rate and discount factor discussed in the main text.

The exploration schedule has a moderate effect. Both decay schedules eventually reach good performance, but slower decay generally produces smoother learning by maintaining exploration for longer. The network size has a relatively small impact: both configurations learn useful policies, suggesting that Acrobot does not require a large function approximator. Replay-buffer size mainly affects stability, with the larger buffer producing smoother learning and fewer performance drops.

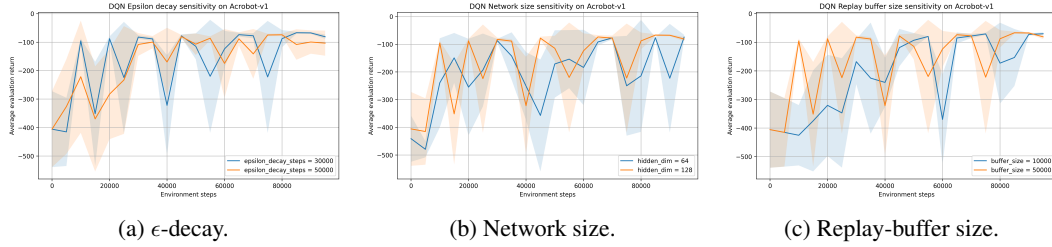


Figure 7: Additional DQN hyperparameter sensitivity experiments on Acrobot-v1.

A.2 DQN on CartPole-v1

Figure 8 presents the remaining DQN sensitivity experiments on CartPole-v1.

Compared with Acrobot, CartPole is less sensitive to exploration scheduling and network size. Both ϵ -decay schedules and both network sizes achieve similar performance. Replay-buffer size has a more noticeable effect: the smaller buffer often performs better because it contains more recent transitions from stronger policies.

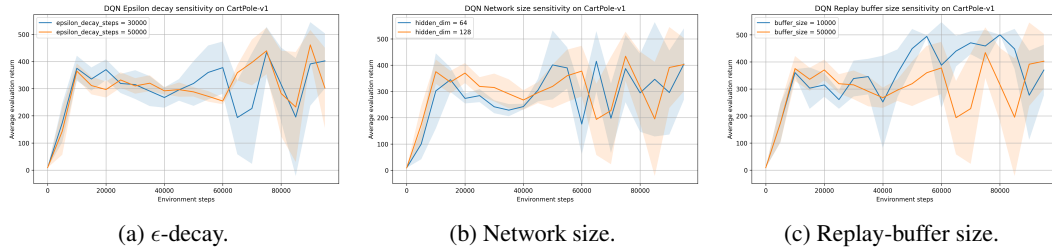
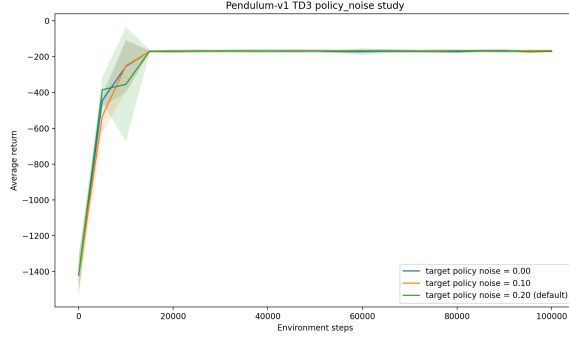


Figure 8: Additional DQN hyperparameter sensitivity experiments on CartPole-v1.

A.3 TD3 on Pendulum-v1

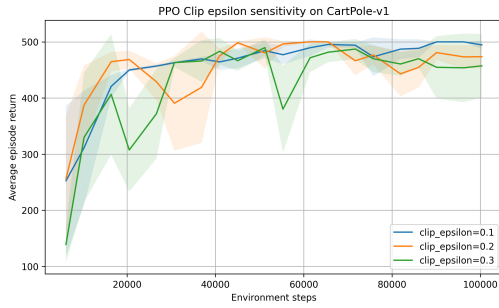
Figure 9 presents the remaining TD3 sensitivity experiment on Pendulum-v1. Unlike policy update frequency and exploration noise, target policy noise had only a weak effect on performance. All tested values converged to similar final returns, showing that target policy smoothing was not the limiting factor in this environment. This is likely because Pendulum-v1 is a dense-reward and relatively low-dimensional continuous-control task, so TD3 was able to learn stable critic estimates for different smoothing values.



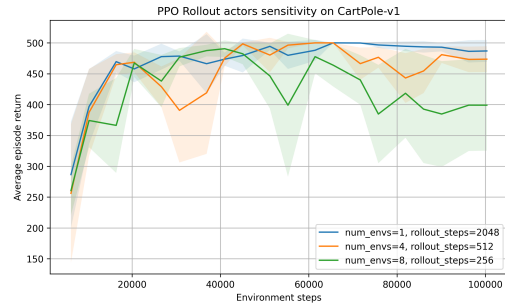
(a) Target policy noise.

Figure 9: Additional TD3 hyperparameter sensitivity experiment on Pendulum-v1.

A.4 PPO on Cartpole-v1



(a) Clipping parameter.



(b) Rollout actors.

Figure 10: PPO hyperparameter sensitivity experiments on CartPole-v1. Each curve is averaged over three seeds.

Figure 10a shows that PPO is not highly sensitive to the clipping parameter on CartPole. All three values, $\epsilon \in \{0.1, 0.2, 0.3\}$, eventually reach high returns close to 500. The smallest value, $\epsilon = 0.1$, is slightly more conservative and gives the most stable final behavior, while $\epsilon = 0.3$ shows larger fluctuations, including a drop around 55,000 steps. This is consistent with the role of PPO clipping: smaller clipping limits policy updates more strongly, while larger clipping allows faster but potentially less stable changes.

The rollout-actor experiment in Figure 10b compares different numbers of parallel actors while keeping the total PPO batch size fixed at 2,048. Using one actor gives the most stable and strongest final performance. Using four actors also performs well, but with more oscillation. Using eight actors is less stable and ends lower than the other settings. Since the batch size is fixed, increasing the number of actors reduces the rollout length per actor. This gives more parallel diversity but shorter temporal segments, which may weaken advantage estimation on CartPole. Therefore, more actors are not automatically better when the total batch size is unchanged.